

```
class Date {
public:
    Date(int month, int day, int year);
    ...
};
```

乍见之下这个接口通情达理（至少在美国如此），但它的客户很容易犯下至少两个错误。第一，他们也许会以错误的次序传递参数：

```
Date d(30, 3, 1995);    //喔欧! 应该是 "3,30" 而不是 "30,3"
```

第二，他们可能传递一个无效的月份或天数：

```
Date d(2, 30, 1995);    //喔欧! 应该是 "3,30" 而不是 "2,30"
```

（上个例子也许看起来很蠢，但别忘了，键盘上的 2 就在 3 旁边。打岔一个键的情况并不是太罕见。）

许多客户端错误可以因为导入新类型而获得预防。真的，在防范“不值得拥有的代码”上，类型系统（**type system**）是你的主要同盟国。既然这样，就让我们导入简单的外覆类型（**wrapper types**）来区别天数、月份和年份，然后于 `Date` 构造函数中使用这些类型：

```
struct Day {
explicit Day(int d)
    : val(d) { }
int val;
};

struct Month {
explicit Month(int m)
    : val(m) { }
int val;
};

struct Year {
explicit Year(int y)
    : val(y) { }
int val;
};

class Date {
public:
    Date(const Month& m, const Day& d, const Year& y);
    ...
};

Date d(30, 3, 1995);                //错误!不正确的类型
Date d(Day(30), Month(3), Year(1995)); //错误!不正确的类型
Date d(Month(3), Day(30), Year(1995)); //OK, 类型正确
```

令 `Day`, `Month` 和 `Year` 成为成熟且经充分锻炼的 **classes** 并封装其内数据，比简单使用上述的 **structs** 好（见条款 22）。但即使 **structs** 也已经足够示范：明智而审慎地导入新类型对预防“接口被误用”有神奇疗效。

一旦正确的类型就定位, 限制其值有时候是通情达理的。例如一年只有 12 个有效月份, 所以 Month 应该反映这一事实。办法之一是利用 enum 表现月份, 但 enums 不具备我们希望拥有的类型安全性, 例如 enums 可被拿来当一个 ints 使用 (见条款 2)。比较安全的解法是预先定义所有有效的 Months:

```
class Month {
public:
    static Month Jan() { return Month(1); }      //函数, 返回有效月份。
    static Month Feb() { return Month(2); }      //稍后解释为什么。
    ...                                           //这些是函数而非对象。
    static Month Dec() { return Month(12); }
    ...                                           //其他成员函数
private:
    explicit Month(int m);                       //阻止生成新的月份。
    ...                                           //这是月份专属数据。
};

Date d(Month::Mar(), Day(30), Year(1995));
```

如果“以函数替换对象, 表现某个特定月份”让你觉得诡异, 或许是因为你忘记了 non-local static 对象的初始化次序有可能出问题。建议阅读条款 4 恢复记忆。

预防客户错误的另一个办法是, 限制类型内什么事可做, 什么事不能做。常见的限制是加上 const。例如条款 3 曾经说明为什么“以 const 修饰 operator* 的返回类型”可阻止客户因“用户自定义类型”而犯错:

```
if (a * b = c) ...      //喔欧, 原意其实是要做一次比较动作!
```

下面是另一个一般性准则“让 types 容易被正确使用, 不容易被误用”的表现形式: “除非有好理由, 否则应该尽量令你的 types 的行为与内置 types 一致”。客户已经知道像 int 这样的 type 有些什么行为, 所以你应该努力让你的 types 在合样合理的前提下也有相同表现。例如, 如果 a 和 b 都是 ints, 那么对 a*b 赋值并不合法, 所以除非你有好的理由与此行为分道扬镳, 否则应该让你的 types 也有相同的表现。是的, 一旦怀疑, 就请拿 ints 做范本。

避免无端与内置类型不兼容, 真正的理由是为了提供行为一致的接口。很少有其他性质比得上“一致性”更能导致“接口容易被正确使用”, 也很少有其他性质比得

上“不一致性”更加剧接口的恶化。STL 容器的接口十分一致(虽然不是完美地一致), 这使它们非常容易被使用。例如每个 STL 容器都有一个名为 `size` 的成员函数, 它会告诉调用者目前容器内有多少对象。与此对比的是 Java, 它允许你针对数组使用 `length property`, 对 Strings 使用 `length method`, 而对 Lists 使用 `size method`; .NET 也一样混乱, 其 Arrays 有个 `property` 名为 `Length`, 其 ArrayLists 有个 `property` 名为 `Count`。有些开发人员会以为整合开发环境 (integrated development environments, IDEs) 能使这一般不一致性变得不重要, 但他们错了。不一致性对开发人员造成的心理和精神上的摩擦与争执, 没有任何一个 IDE 可以完全抹除。

任何接口如果要求客户必须记得做某些事情, 就是有着“不正确使用”的倾向, 因为客户可能会忘记做那件事。例如条款 13 导入了一个 `factory` 函数, 它返回一个指针指向 `Investment` 继承体系内的一个动态分配对象:

```
Investment* createInvestment();           //来自条款 13; 为求简化暂略参数。
```

为避免资源泄漏, `createInvestment` 返回的指针最终必须被删除, 但那至少开启了两个客户错误机会: 没有删除指针, 或删除同一个指针超过一次。

条款 13 表明客户如何将 `createInvestment` 的返回值存储于一个智能指针如 `auto_ptr` 或 `tr1::shared_ptr` 内, 因而将 `delete` 责任推给智能指针。但万一客户忘记使用智能指针怎么办? 许多时候, 较佳接口的设计原则是先发制人, 就令 `factory` 函数返回一个智能指针:

```
std::tr1::shared_ptr<Investment> createInvestment();
```

这便实质上强迫客户将返回值存储于一个 `tr1::shared_ptr` 内, 几乎消弭了忘记删除底部 `Investment` 对象(当它不再被使用时)的可能性。

实际上, 返回 `tr1::shared_ptr` 让接口设计者得以阻止一大群客户犯下资源泄漏的错误, 因为就如条款 14 所言, `tr1::shared_ptr` 允许当智能指针被建立起来时指定一个资源释放函数(所谓删除器, “`deleter`”)绑定于智能指针身上(`auto_ptr` 就没有这种能耐)。

假设 `class` 设计者期许那些“从 `createInvestment` 取得 `Investment*` 指针”的客户将该指针传递给一个名为 `getRidOfInvestment` 的函数, 而不是直接在它身上动刀(使用 `delete`)。这样一个接口又开启通往另一个客户错误的大门, 该错误是“企图使用错误的资源析构机制”(也就是拿 `delete` 替换 `getRidOfInvestment`)。

`createInvestment` 的设计者可以针对此问题先发制人：返回一个“将 `getRidOfInvestment` 绑定为删除器 (deleter)”的 `trl::shared_ptr`。

`trl::shared_ptr` 提供的某个构造函数接受两个实参：一个是被管理的指针，另一个是引用次数变成 0 时将被调用的“删除器”。这启发我们创建一个 `null trl::shared_ptr` 并以 `getRidOfInvestment` 作为其删除器，像这样：

```
std::trl::shared_ptr<Investment>          //企图创建一个 null shared_ptr
pInv(0, getRidOfInvestment);              //并携带一个自定的删除器。
                                           //此式无法通过编译。
```

啊呀，这不是有效的 C++。`trl::shared_ptr` 构造函数坚持其第一参数必须是个指针，而 0 不是指针，是个 `int`。是的，它可被转换为指针，但在此情况下并不够好，因为 `trl::shared_ptr` 坚持要一个不折不扣的指针。转型 (cast) 可以解决这个问题：

```
std::trl::shared_ptr<Investment>          //建立一个 null shared_ptr 并以
pInv( static_cast<Investment*>(0),        //getRidOfInvestment 为删除器；
getRidOfInvestment);                     //条款 27 提到 static_cast
```

因此，如果我们要实现 `createInvestment` 使它返回一个 `trl::shared_ptr` 并夹带 `getRidOfInvestment` 函数作为删除器，代码看起来像这样：

```
std::trl::shared_ptr<Investment> createInvestment()
{
    std::trl::shared_ptr<Investment> retVal(static_cast<Investment*>(0),
                                           getRidOfInvestment);

    retVal = ... ; //令 retVal 指向正确对象
    return retVal;
}
```

当然啦，如果被 `pInv` 管理的原始指针 (raw pointer) 可以在建立 `pInv` 之前先确定下来，那么“将原始指针传给 `pInv` 构造函数”会比“先将 `pInv` 初始化为 `null` 再对它做一次赋值操作”为佳。至于其原因，请见条款 26。

`trl::shared_ptr` 有一个特别好的性质是：它会自动使用它的“每个指针专属的删除器”，因而消除另一个潜在的客户错误：所谓的“cross-DLL problem”。这个问题发生于“对象在动态连接程序库 (DLL) 中被 `new` 创建，却在另一个 DLL 内被 `delete` 销毁”。在许多平台上，这一类“跨 DLL 之 `new/delete` 成对运用”会导致运行期错误。`trl::shared_ptr` 没有这个问题，因为它缺省的删除器是来自“`trl::shared_ptr` 诞生所在的那个 DLL”的 `delete`。这意思是……唔……让我举个例子，如果 `Stock` 派生自 `Investment` 而 `createInvestment` 实现如下：

```
std::tr1::shared_ptr<Investment> createInvestment()
{
    return std::tr1::shared_ptr<Investment>(new Stock);
}
```

返回的那个 `tr1::shared_ptr` 可被传递给任何其他 DLLs, 无需在意 "cross-DLL problem"。这个指向 `Stock` 的 `tr1::shared_ptr`s 会追踪记录“当 `Stock` 的引用次数变成 0 时该调用的那个 DLL's delete”。

本条款并非特别针对 `tr1::shared_ptr`, 而是为了“让接口容易被正确使用, 不容易被误用”而设。但由于 `tr1::shared_ptr` 如此容易消除某些客户错误, 值得我们核计其使用成本。最常见的 `tr1::shared_ptr` 实现品来自 `Boost` (见条款 55)。Boost 的 `shared_ptr` 是原始指针 (raw pointer) 的两倍大, 以动态分配内存作为簿记用途和“删除器之专属数据”, 以 `virtual` 形式调用删除器, 并在多线程程序修改引用次数时蒙受线程同步化 (thread synchronization) 的额外开销。(只要定义一个预处理器符号就可以关闭多线程支持)。总之, 它比原始指针大且慢, 而且使用辅助动态内存。在许多应用程序中这些额外的执行成本并不显著, 然而其“降低客户错误”的成效却是每个人都看得到。

请记住

- 好的接口很容易被正确使用, 不容易被误用。你应该在你的所有接口中努力达成这些性质。
- “促进正确使用”的办法包括接口的一致性, 以及与内置类型的行为兼容。
- “阻止误用”的办法包括建立新类型、限制类型上的操作, 束缚对象值, 以及消除客户的资源管理责任。
- `tr1::shared_ptr` 支持定制型删除器 (custom deleter)。这可防范 DLL 问题, 可被用来自动解除互斥锁 (mutexes; 见条款 14) 等等。

条款 19: 设计 class 犹如设计 type

Treat class design as type design.

C++ 就像在其他 OOP（面向对象编程）语言一样，当你定义一个新 class，也就定义了一个新 type。身为 C++ 程序员，你的许多时间主要用来扩张你的类型系统（type system）。这意味你并不只是 class 设计者，还是 type 设计者。重载（overloading）函数和操作符、控制内存的分配和归还、定义对象的初始化和终结……全都在你手上。因此你应该带着和“语言设计者当初设计语言内置类型时”一样的谨慎来研讨 class 的设计。

设计优秀的 classes 是一项艰巨的工作，因为设计好的 types 是一项艰巨的工作。好的 types 有自然的语法，直观的语义，以及一或多个高效实现品。在 C++ 中，一个不良规划下的 class 定义恐怕无法达到上述任何一个目标。甚至 class 的成员函数的效率都有可能受到它们“如何被声明”的影响。

那么，如何设计高效的 classes 呢？首先你必须了解你面对的问题。几乎每一个 class 都要求你面对以下提问，而你的回答往往导致你的设计规范：

- **新 type 的对象应该如何被创建和销毁？** 这会影响到你的 class 的构造函数和析构函数以及内存分配函数和释放函数（`operator new`, `operator new[]`, `operator delete` 和 `operator delete[]`——见第 8 章）的设计，当然前提是如果你打算撰写它们。
- **对象的初始化和对象的赋值该有什么样的差别？** 这个答案决定你的构造函数和赋值（*assignment*）操作符的行为，以及其间的差异。很重要的是别混淆了“初始化”和“赋值”，因为它们对应于不同的函数调用（见条款 4）。
- **新 type 的对象如果被 *passed by value*（以值传递），意味着什么？** 记住，*copy* 构造函数用来定义一个 type 的 *pass-by-value* 该如何实现。
- **什么是新 type 的“合法值”？** 对 class 的成员变量而言，通常只有某些数值集是有效的。那些数值集决定了你的 class 必须维护的约束条件（*invariants*），也就决定了你的成员函数（特别是构造函数、赋值操作符和所谓“setter”函数）必须进行的错误检查工作。它也影响函数抛出的异常、以及（极少被使用的）函数异常明细列（*exception specifications*）。